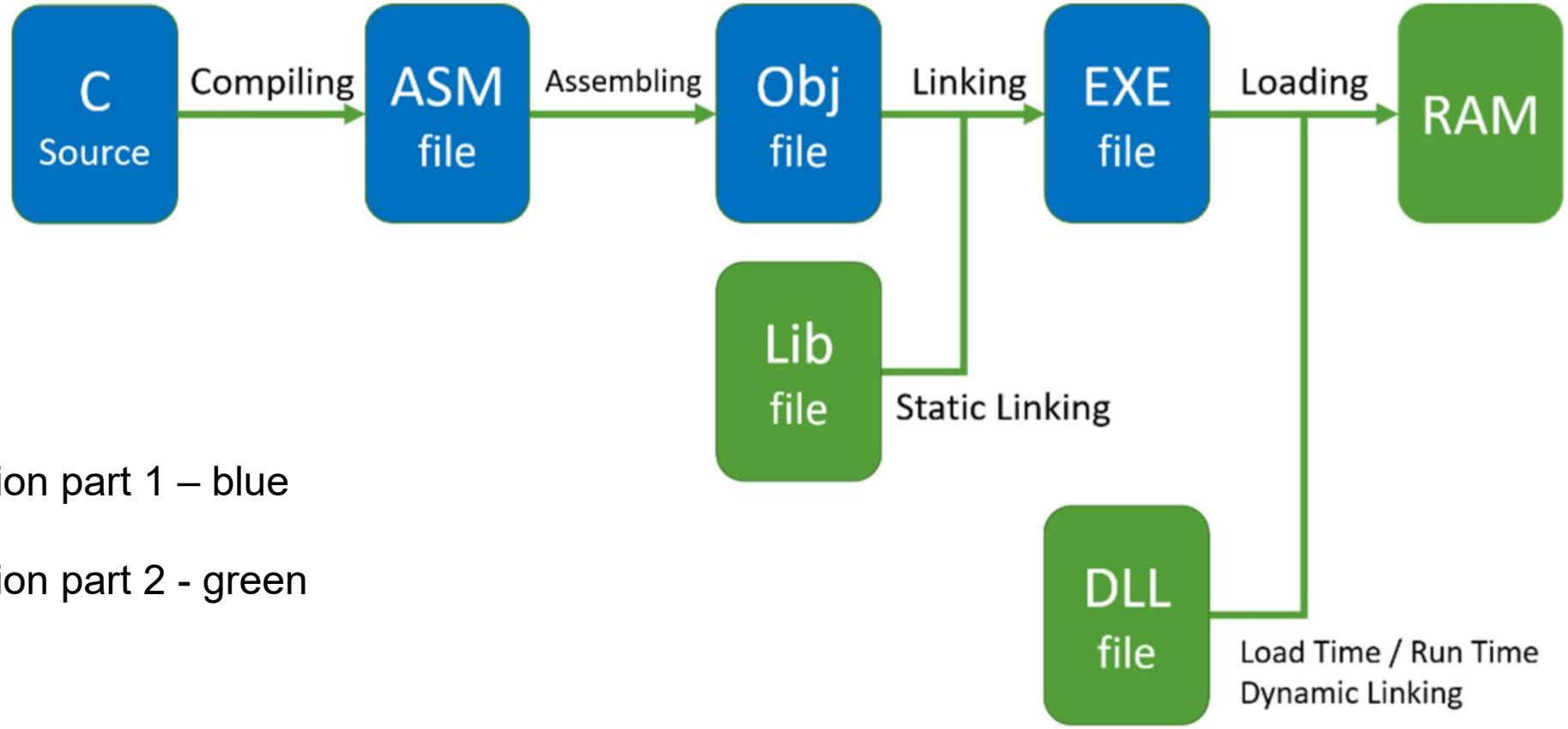
OS for Reverse Engineering

Compiling, assembling, linking, DLL, PE format

Barak Gonen

The complete process



Presentation part 1 – blue

Presentation part 2 - green

Part 1 - Compiling, Assembling, Linking

- Main_file.c , my_funcs.c
- Compiled with mingw

main_file.c

```c
#include <stdio.h>
#include "my_funcs.h"

int main()
{
    static int a = 2;
    static int b = 3;
    int c;
    c = my_add(a, b);
    printf("%d", c);
    return 0;
}
```

my_funcs.c

```c
int my_add(int a, int b)
{
    static int one = 1;
    return a+b+one;
}
```

Compiling

```
movl    _b.2064, %edx
movl    _a.2063, %eax
movl    %edx, 4(%esp)
movl    %eax, (%esp)
call    _my_add
movl    %eax, 28(%esp)
movl    28(%esp), %eax
movl    %eax, 4(%esp)
movl    $LC0, (%esp)
call    _printf
movl    $0, %eax
leave
```

Assembling

- View of main_file.o
- Addresses are not resolved yet

```
seg000:00000112 8B 15 00 00 00 00     mov     edx, ds:dword_0
seg000:00000118 A1 04 00 00 00        mov     eax, ds:dword_4
seg000:0000011D 89 54 24 04           mov     [esp+4], edx
seg000:00000121 89 04 24              mov     [esp], eax
seg000:00000124 E8 00 00 00 00        call    $+5
seg000:00000129 89 44 24 1C           mov     [esp+1Ch], eax
seg000:0000012D 8B 44 24 1C           mov     eax, [esp+1Ch]
seg000:00000131 89 44 24 04           mov     [esp+4], eax
seg000:00000135 C7 04 24 00 00 00 00  mov     dword ptr [esp], 0
seg000:0000013C E8 00 00 00 00        call    $+5
seg000:00000141 B8 00 00 00 00        mov     eax, 0
seg000:00000146 C9                    leave
```

Obj file

- Machine code
- Addresses that need to be resolved
- Symbol table – names of variables
- Debug info (optional) – names of variables, functions

  Hands on – open the obj file in hex editor and look for debug info

Linking

- Output – lib file or exe file
- Uniting all code / data sections to one section
- In case of exe: address resolution
- Hands on:

  Open IDA and search:

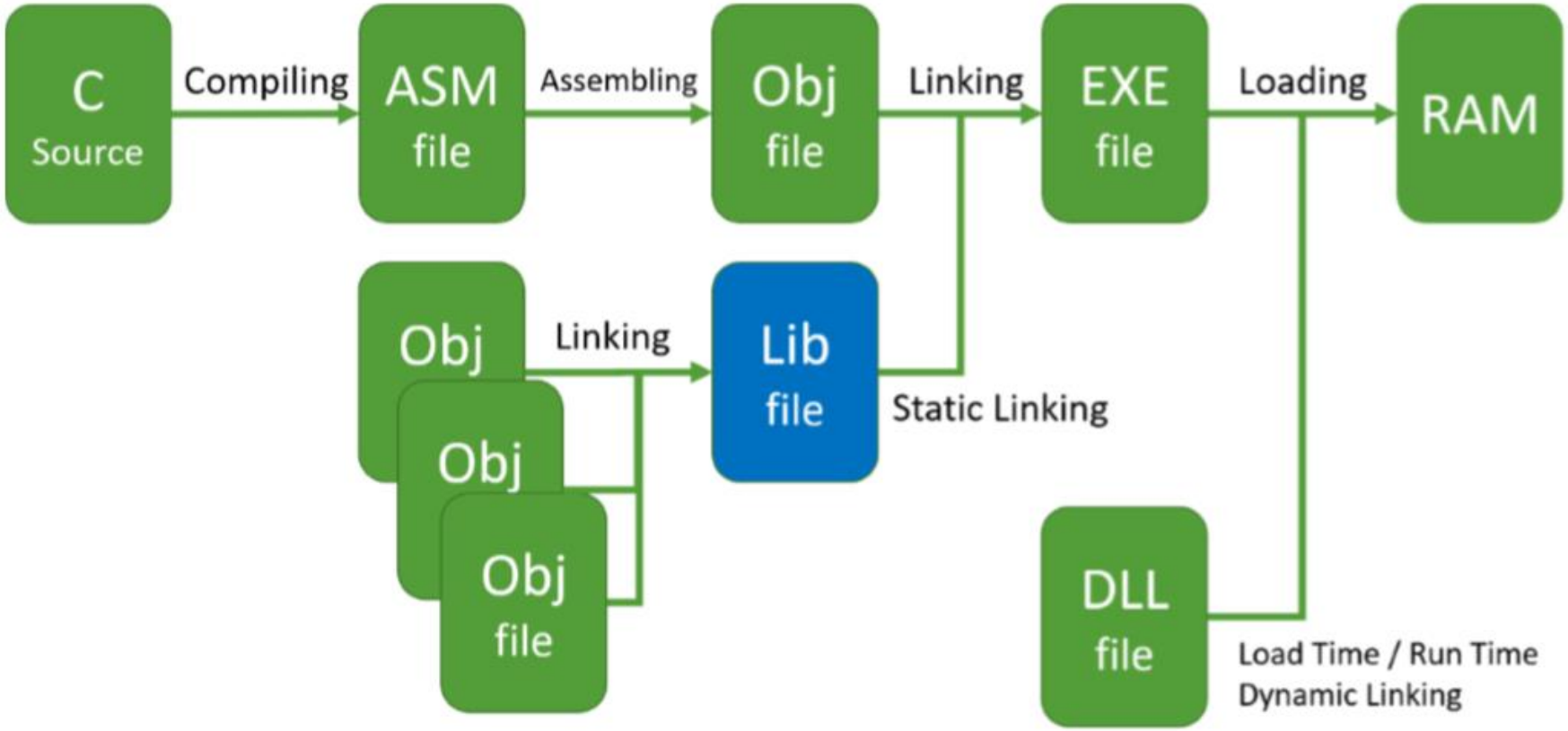
  - main_file code and my_funcs code
  - Variables addresses
  - printf code

EXE file

```
.text:0040141E 8B 15 04 40 40 00      mov     edx, _b_2064
.text:00401424 A1 08 40 40 00         mov     eax, _a_2063
.text:00401429 89 54 24 04            mov     [esp+4], edx
.text:0040142D 89 04 24               mov     [esp], eax
.text:00401430 E8 1F 00 00 00         call    _my_add
.text:00401435 89 44 24 1C            mov     [esp+1Ch], eax
.text:00401439 8B 44 24 1C            mov     eax, [esp+1Ch]
.text:0040143D 89 44 24 04            mov     [esp+4], eax
.text:00401441 C7 04 24 44 50 40 00   mov     dword ptr [esp], offset aD ; "%d"
.text:00401448 E8 73 26 00 00         call    _printf
.text:0040144D B8 00 00 00 00         mov     eax, 0
.text:00401452 C9                     leave

.text:00401457 8B 55 08               mov     edx, [ebp+arg_0]
.text:0040145A 8B 45 0C               mov     eax, [ebp+arg_4]
.text:0040145D 01 C2                  add     edx, eax
.text:0040145F A1 0C 40 40 00         mov     eax, _one_1554
.text:00401464 01 D0                  add     eax, edx
```

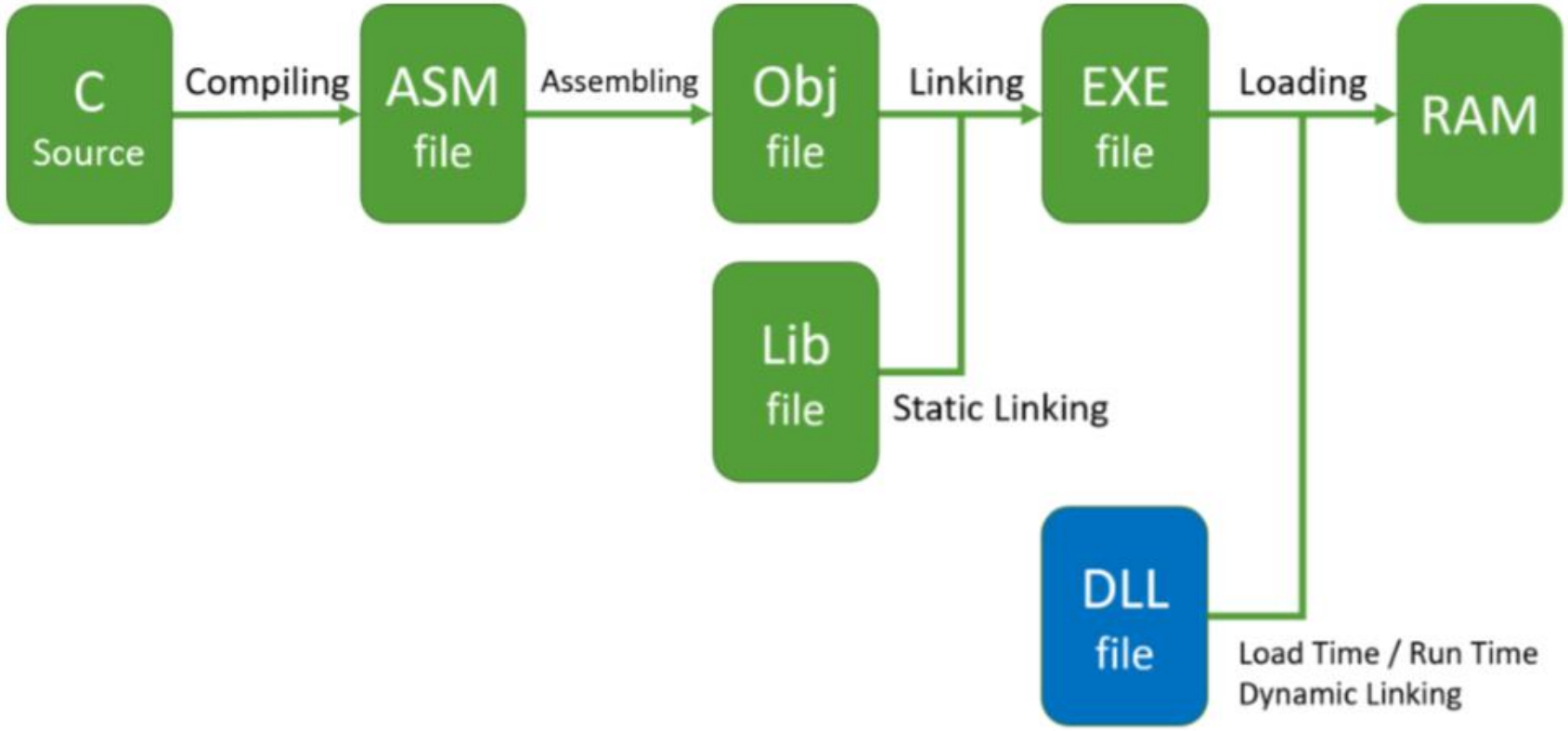
Library, Static Linking



DLL

- Dynamic Link Library
- A code which is shared among processes
  - No need for source code (intellectual property reserved)
  - Loaded to memory only once (saves memory)
- For example
  - user32.dll – Windows management functions for message handling, timers, menus, and communications
  - Kernel32.dll – Low-level operating system functions for memory management and resource handling
- Guess: how many processes currently use user32.dll?

DLL


C
Source
Compiling
ASM
file
Assembling
Obj
file
Linking
EXE
file
Loading
RAM
Lib
file
Static Linking
DLL
file
Load Time / Run Time
Dynamic Linking

Load Time Dynamic Linking

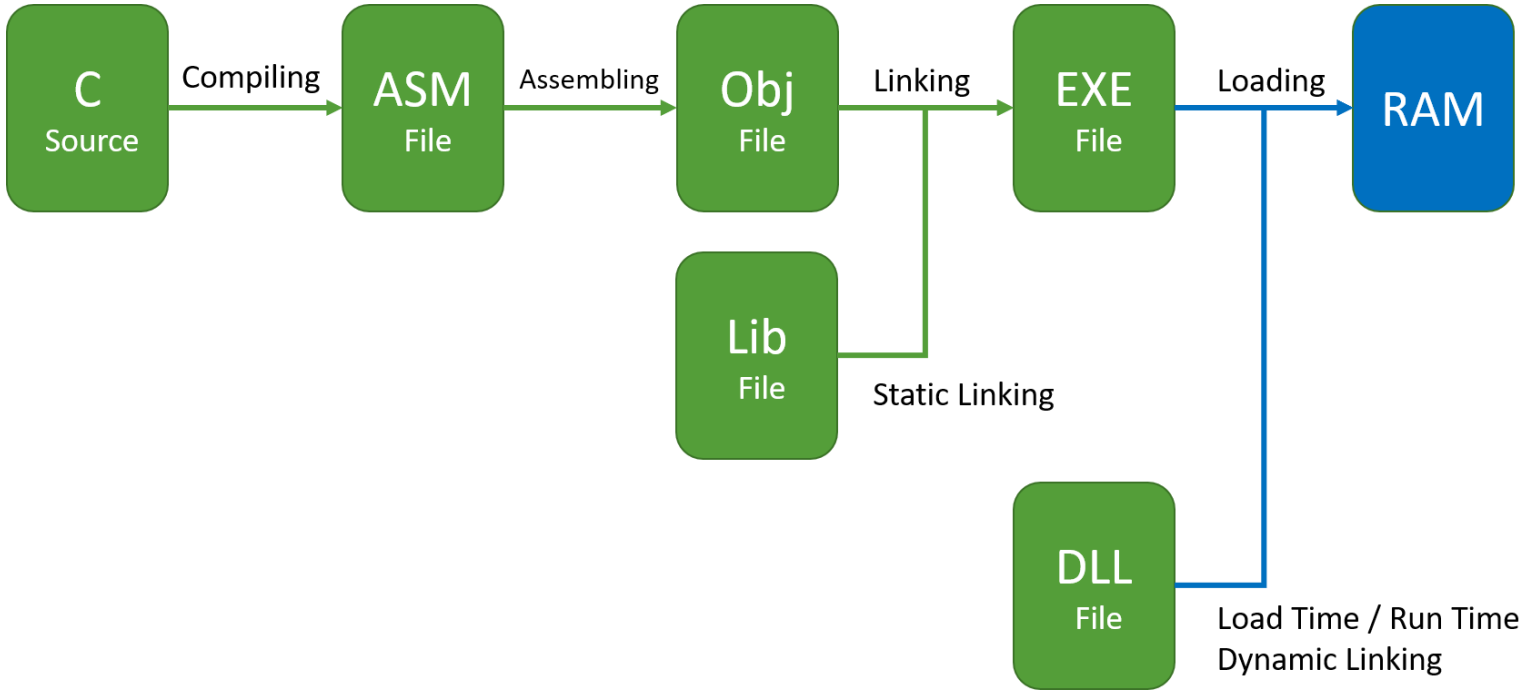
- The exe includes a directive to load the required DLL
- Study from OS book

Run Time Dynamic Linking

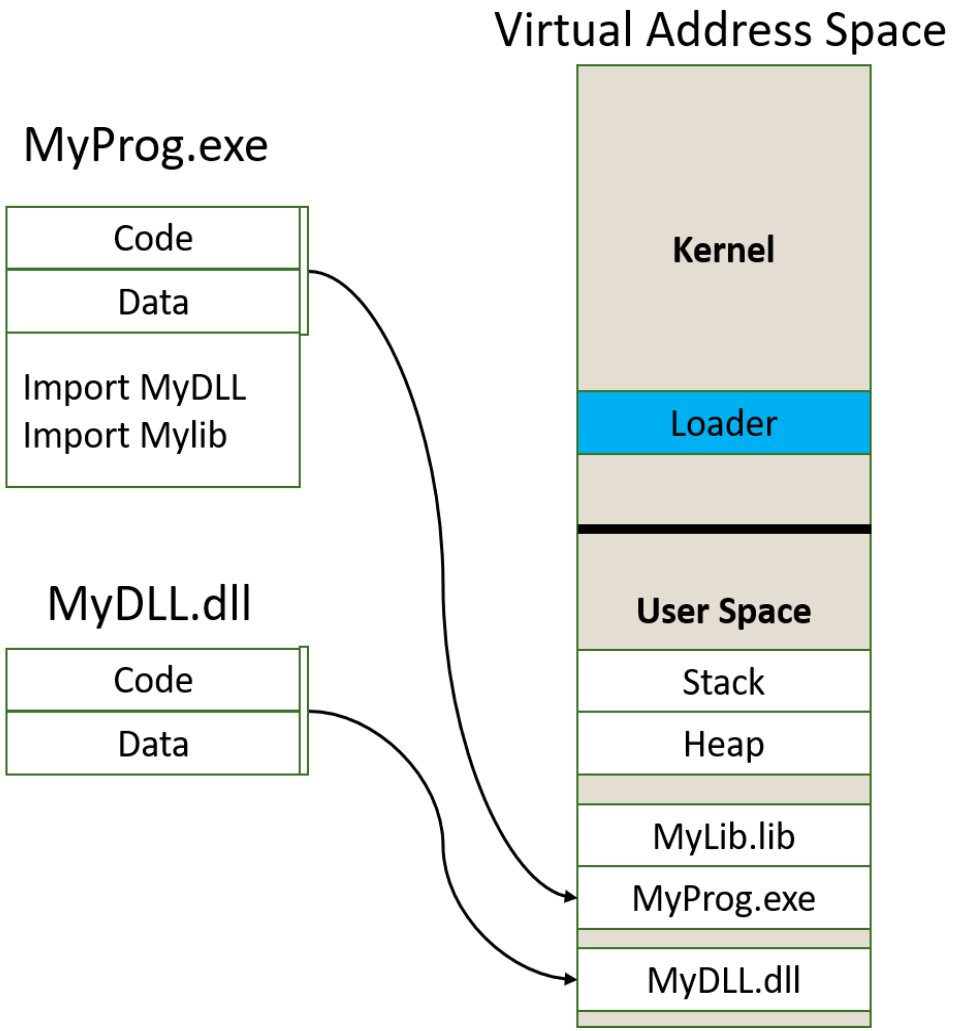
- Usage of WinAPI functions:
  - LoadLibrary – load DLL to memory
  - GetProcAddress – get the address of the required function
- Study from OS book

Part 2 – Loading and running

- The loader
- Format PE



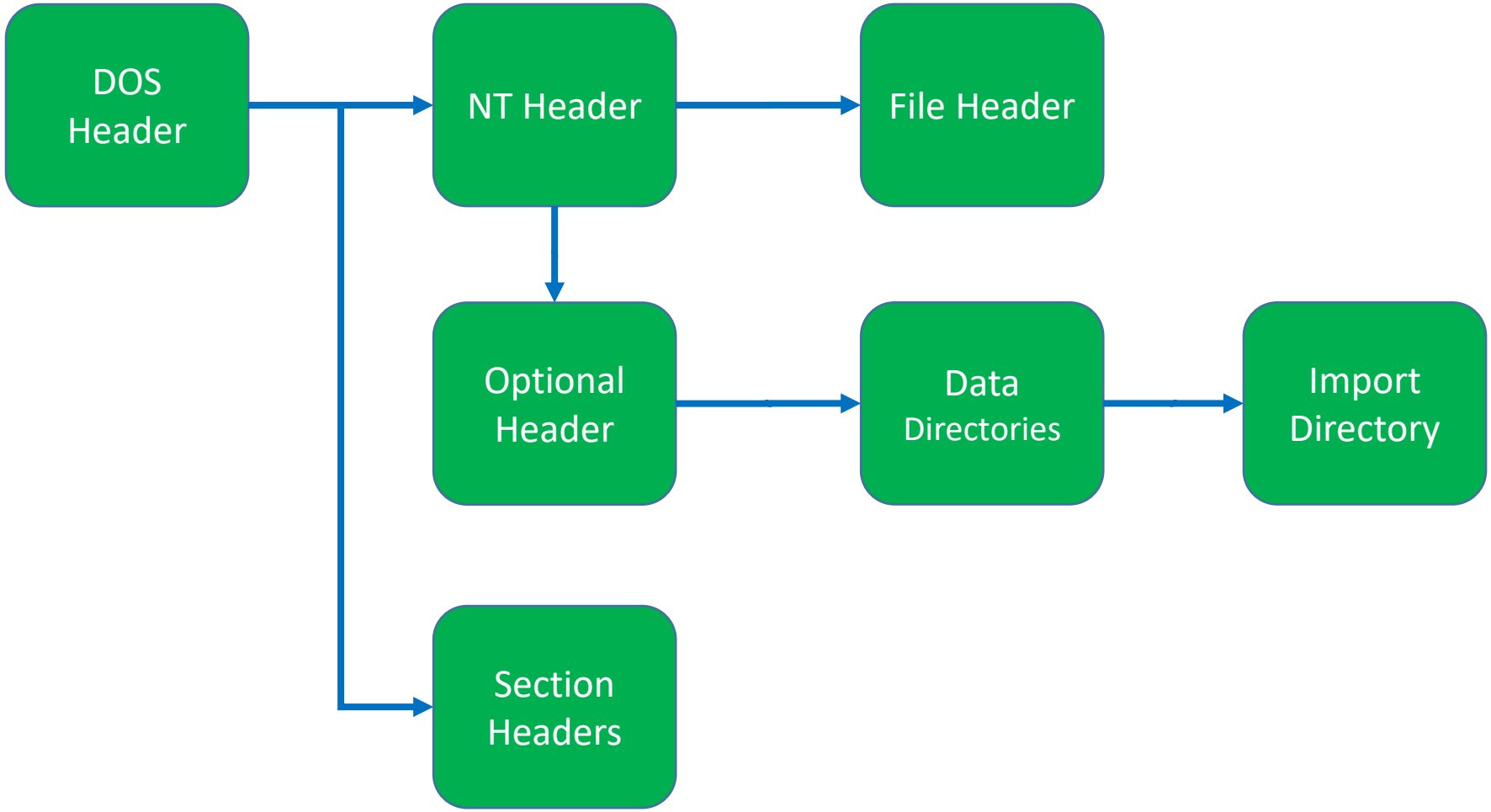
The Role of the Loader



PE Format

- Microsoft format for exe, dll files
- Portable Executable – can compile on one computer, then run on another
- https://ntcore.com/files/ExplorerSuite.exe

PE Format


DOS Header
NT Header
File Header
Optional Header
Data Directories
Import Directory
Section Headers

PE Format

- MZ, DOS stub
- Entry point
- Base address
- Memory sections
- DLL’s to load
- Relocation table
- Import address table
- DLL characteristics

Address Resolution

- This code calls a function in a DLL
- First snippet is before loading, second - after
- Note the address changed

```
.text:004116FE B9 03 B0 41 00     mov     ecx, offset unk_41B003
.text:00411703 E8 FB FA FF FF     call    sub_411203
.text:00411708 8B F4              mov     esi, esp
.text:0041170A FF 15 E0 A0 41 00  call    ds:Share
```

```
009c16fe b903b09c00   mov     ecx, offset usemydll!_AD1F52AD_usemydll.cpp
009c1703 e8fbfaffff   call    usemydll!ILT+510(__CheckForDebuggerJustMyCo
009c1708 8bf4         mov     esi, esp
009c170a ff15e0a09c00 call    dword ptr [usemydll!_imp__Share (009ca0e0)]
```

How debuggers work

- Google code “0xCC”
- Hands on IDA + Process Explorer